College of Engineering and Information Technology

# Flying Radar: A Drone-Based Software-Defined Radar

---

**Commercial In Confidence**

ENGI 4900: Final Report

13 Nov 2026

36/36 Pages

# Summary of AI Use

Categories of artificial intelligence (AI) tool use in this assignment are as indicated in the following table.

| **AI tool use category** | **Used** |
|---|---|
| Spelling | ✓ |
| Grammar | ✓ |
| Other copy editing | |
| Literature review support | |
| Visuals | |
| Substantive editing | |
| Idea generation | |
| Translation | |
| Research technical support | ✓ |
| No AI tool was used | |

# Acknowledgements

# Contents

**Summary of AI Use** i

**Acknowledgements** ii

**List of Figures** iv

**List of Tables** v

**List of Acronyms** vii

**Contribution Statement** viii

**4 Methods, Results and Discussion** x

4.1 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements . x

4.1.1 Introduction . . . . . x

4.1.2 Methodology . . . . . x

4.1.3 Results and Discussion . . . . . xiv

4.1.4 Summary . . . . . xiv

**References** xiv

**Statement of AI Use** xv

**A Objective 4: Supporting Material** xvi

A.1 SSH Setup . . . . . xvi

A.2 GPIO Packages Install . . . . . xvi

A.3 Virtual Environment and `rpi-lgpio` Install . . . . . xvi

A.4 GPIO Read Script . . . . . xvi

A.5 Telemetry Data Visualisation Script . . . . . xvii

# List of Figures

4.1 Activation signal flow starting at the pilot transmitter and ending at the software-defined radar (SDR). . . . . x

4.2 Control system development cycle . . . . . xi

4.3 Flight path and radar capture locations from a test flight plotted in the data visualisation script. . . . . xiv

# List of Tables

4.1 MAVLink messages requested from the flight controller and their use. . . . . xiii

# List of Acronyms

| Acronym | Definition |
|---|---|
| 3D | three dimensional |
| AC | alternating current |
| AI | artificial intelligence |
| API | application programming interface |
| ASA | acrylonitrile styrene acrylate |
| AST | Advanced Systems & Technologies |
| CLI | command-line interface |
| CPU | central processing unit |
| C-UAS | counter-uncrewed aerial system |
| CAD | computer-aided design |
| COTS | commercial off-the-shelf |
| CW | continuous wave |
| DC | direct current |
| DEM | digital elevation model |
| FMCW | frequency modulated continuous wave |
| GPIO | general purpose input/output |
| GPR | ground penetrating radar |
| GPS | global positioning system |
| HALE | high-altitude long-endurance |
| IDE | integrated development environment |
| IED | improvised explosive device |
| InSAR | interferometric synthetic aperture radar |
| IQ | in-phase and quadrature |
| JSON | JavaScript Object Notation |

| | |
|---|---|
| JST | Japan solderless terminal |
| LiPo | lithium-polymer |
| PC | personal computer |
| PCB | printed circuit board |
| PID | product identifier |
| PLA | polylactic acid |
| PWM | pulse-width modulation |
| RAM | random-access memory |
| RC | radio control |
| RF | radio frequency |
| SAR | synthetic aperture radar |
| SBC | single-board computer |
| SD | secure digital |
| SDR | software-defined radar |
| SI | International System of Units |
| SLD | single-line diagram |
| SSH | secure shell |
| UAV | uncrewed aerial vehicle |
| USB | universal serial bus |
| VID | vendor identifier |

# Contribution Statement

This contribution statement records the specific contribution of each team member to the technical work and preparation of this progress report. Contributions are grouped according to the simplified CRediT categories of Approach, Technical output, Writing, and Project management.

Table CS1: Declaration of group contributions to the report.

| Author | Team Role | Approach | Technical Output | Writing | Project Management |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

Table CS2: Acknowledgement of AI Tool Use

| **Group member name** | **Specific sub-sections or paragraphs where AI was used** | **AI Tool Use Category/ies** |
|---|---|---|

Table CS3: Group Member Comments

| **Group member name** | **Comments** |
|---|---|

By signing below students are acknowledging that the contributions recorded above are a true representation of the contribution made by each individual.

Table CS4: Signature Acknowledgement of Contribution

| **Name** | **Student ID** | **Signature** |
|---|---|---|

# 4 Methods, Results and Discussion

## 4.1 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements

### 4.1.1 Introduction

To synchronise flight telemetry and radar measurements, a pilot-initiated control signal must be interpreted, processed, and transmitted through the system. The signal travels from the pilot transmitter to the receiver, then to the flight controller, and finally to the single-board computer (SBC), which triggers SDR logging. This signal path is shown in Figure 4.1. Synchronising the radar captures and the telemetry is critical, as each radar capture needs a position and attitude before it can be georeferenced or used in later processing.

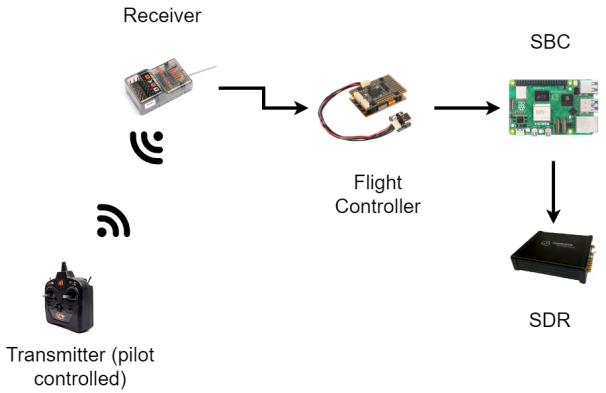



Figure 4.1: Activation signal flow starting at the pilot transmitter and ending at the SDR.

The work done in this objective included researching and procuring a suitable SBC, configuring the flight controller, designing software to interface with the flight controller and radar, and implementing a post-processing script. The requirements of the system was the ability for a pilot to trigger the radar capture directly from a switch on the transmitter, and that every capture is synchronised to the telemetry for processing. The final system was a MAVLink-based Python program on board a Raspberry Pi 5 that interfaced with both the radar and the flight controller.

### 4.1.2 Methodology

#### Hardware and Setup

First, requirements of the SBC were identified. The hardware requirements for the SBC were derived from the SDR datasheet, which specifies the need for a high-speed USB 2.0 interface (480 Mb/s) [**?**]. Additionally, the SBC should provide sufficient processing capability to support potential future expansion of onboard processing capability, placing requirements on central processing unit (CPU) performance and random-access memory (RAM) capacity. Consequently, a Raspberry Pi 5 with 16 GB of RAM was selected and procured. In addition to satisfying the hardware requirements, Raspberry Pi devices were selected because of their well-supported software ecosystem and prior use in drone-mounted radar research applications [**?**, **?**, **?**, **?**]. A protective case and active cooler were also acquired to secure the device on board the drone and manage CPU temperature during SDR operation.

The Raspberry Pi 5 was physically assembled and configured with a lightweight Ubuntu 22.04 operating system. A remote development workflow was then configured to develop and test the

Python script used to read the general purpose input/output (GPIO) pin. Native development on the Raspberry Pi was avoided due to its lower performance compared with a desktop computer. Instead, secure shell (SSH) was configured and integrated with an integrated development environment (IDE) to enable remote development. The setup commands are shown in Appendix A.1. To allow the Raspberry Pi to read GPIO pins, the required GPIO packages were installed. The setup commands are shown in Appendix A.2. One of the packages required a Python virtual environment. The setup and installation of this package and a Python virtual environment is shown in Appendix A.3. After installing the packages, a Python script was created to read a GPIO pin and respond when an external component pulls the signal high. The script is shown in Appendix A.4. To verify the program, an Arduino Uno R3 was used to generate an input signal for a GPIO pin. After the setup was confirmed working, the objective moved toward the development stage.

### Development Approach

The development approach followed the diagram shown in Figure 4.2. Section 4.1.2 detailed the SBC setup and the GPIO read. The next development of the objective read the flight controller's PWM signal directly from the GPIO pins, afterward this was revised for MAVLink to be used as the interfacing protocol in place of the GPIO pins.

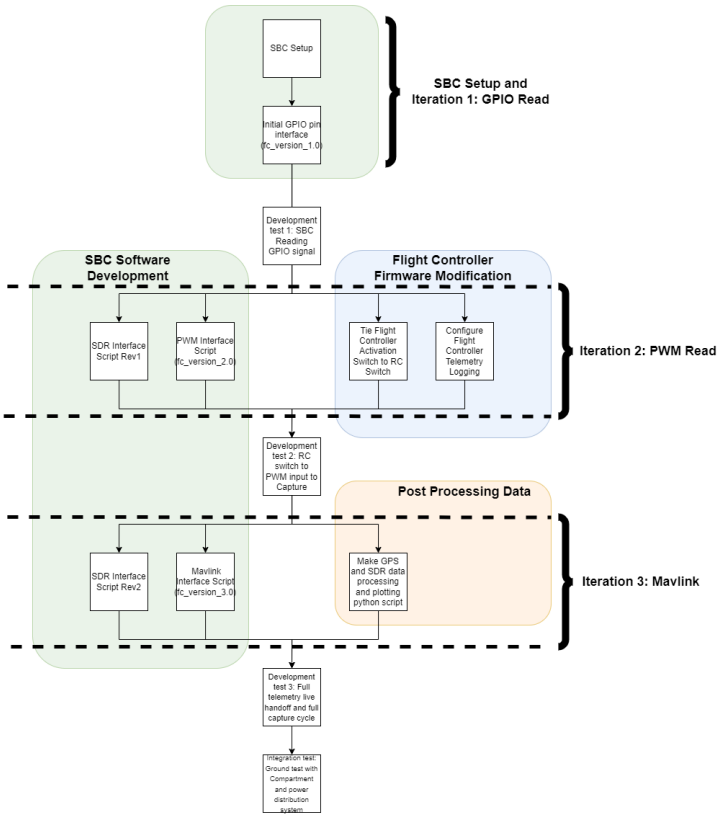



Figure 4.2: Control system development cycle

The flight controller uses pulse-width modulation (PWM) as the method to encode data to be sent. The protocol encodes data in the pulse width of square wave signals and is typically used for servo control (citation needed). Whether a signal was logic HIGH or logic LOW was determined by the pulse width of the transmission. For the flight controller signal, a pulse width below 1300 $\mu s$ was logic LOW, and a pulse width above 1700 $\mu s$ was logic HIGH. The gap between LOW and HIGH acts as a dead-band for noisy signals.

Initially, the PWM signal was read directly from servo output pins into the GPIO pins on the SBC. This was eventually replaced with a direct USB connection using the MAVLink protocol to read in the radio control (RC) channel signal directly from the flight controller communications, instead of reading a servo output. Doing this simplified physical wiring greatly as with MAVLink all communication is handled over a single universal serial bus (USB) connection. The GPIO activation signal was also dependent on the safety switch status of the drone; it had to be off for an output to occur. This issue is not present in the MAVLink setup.

After the MAVLink software was created, a separate post-processing script in Python to be used on a desktop PC to process the data was developed. This concluded the development cycle of the control system.

### Flight Controller Configuration

The flight controller needed to be configured to read in a signal from the transmitter and tie that signal to a specific channel. On the transmitter in possession, there were 8 physical switches. The switch that was deemed suitable was tied to RC channel 15 using the transmitter settings. RC Channel 15 was then tied to servo output 8. After configuring, the output signal was scoped with an oscilloscope and confirmed working as intended. After the system swapped to using MAVLink, the servo output was no longer used and PWM was instead read from the MAVLink connection.

It was also found that when the host PC connects to the flight controller, the flight controller makes a .tlog file and continues to write to the .tlog file until the connection is terminated. The telemetry log only records global positioning system (GPS) when the drone is armed. This setting did not suit the project, as the drone needed to be disarmed for the majority of the project. To enable GPS logging while disarmed, a parameter was changed in the config file on the flight controller. Eventually the .tlog file was replaced by a .csv file on board the SBC as it was able to read all relevant telemetry status' directly. However, the .tlog file serves as a cross-check between the .csv file and the drone's reported status from the flight controller.

### Final System

A single python 3 program on the SBC handled both the flight controller and the SDR in one loop. The code was split by device, a `Drone` class, which handled the flight controller interface, a `Radar` class, which handled the radar interface, and a small `main.py` script to run the overall program.

The benefit to the program being one system was the fact that all the timing was able to use a single clock, which was `runtime` (seconds elapsed since the start of the program). Both the telemetry log and the radar used this value, and shared the same time base which is what made synchronisation possible.

*Mavlink Interface*

The SBC interfaced with the drone via Mavlink, a communication protocol used to interface between flight controllers and desktop computers, which the Raspberry Pi 5 running Ubuntu resembles in architecture. The physical connection was over USB, and the software interface was handled by the `pymavlink` python package.

The script blocked until it received a `HEARTBEAT` signal which confirmed the link and gave the target system and component IDs to be used by later commands. Instead of relying on default stream rates, the script sent `MAV_CMD_SET_MESSAGE_INTERVAL` commands to request each message

at a fixed rate. The message IDs of the key messages is shown in Table 4.1.

Table 4.1: MAVLink messages requested from the flight controller and their use.

| Message | ID | Rate | Used for |
|---|---|---|---|
| `GPS_RAW_INT` | 24 | 1 Hz | Fix type, satellites, latitude, longitude |
| `GLOBAL_POSITION_INT` | 33 | 5 Hz | Altitude (AMSL and relative), groundspeed |
| `ATTITUDE` | 30 | 5 Hz | Roll, pitch, yaw |
| `BATTERY_STATUS` | 147 | 1 Hz | Voltage, current, remaining capacity |
| `VFR_HUD` | 74 | 2 Hz | Requested but not parsed |
| `RC_CHANNELS` | 65 | 5 Hz | Channel 15 (radar trigger) |
| `HEARTBEAT` | 0 | Default | Armed state, flight mode |

On each loop in the `main.py` file, `update()` drained every queued message with non-blocking `recv_match(blocking=False)`, and so the program was never stalled by a slow link as the stored value was always the latest received. Raw Mavlink fields were converted to International System of Units (SI) units upon reception, including converting latitude and longitude from degE7 to degrees, altitude from millimetres to metres, and voltage from millivolts to volts. The armed status of the drone was read from the `MAV_MODE_FLAG_SAFETY_ARMED` bit of `HEARTBEAT.base_mode`, and the flight mode was decoded with `mode_string_v10`. The `gps_ready` state was changed based on whether a fix to three or more satellites was present.

The radar was triggered from a switch on the pilot's transmitter which was mapped to channel 15. However, instead of reading in a PWM signal output from the flight controller, the signal was read through the `RC_CHANNELS` MAVlink message. The trigger logic remained the same; rising edge detection. A capture was requested by the main script to the radar only when the pulse width was above 1800 $\mu s$. The capture logic was sticky; the switch had to be seen as low before a new capture could be requested (which was a pulse width below 1200 $\mu s$). The sticky logic prevented one switch from firing repeatedly, ignored the noise in the invalid range, and prevented a switch left high at startup from requesting a capture. Both pulse width parameters can be changed easily in the config file. Buffering was not used; if the radar was still busy, the request was rejected and a message printed, instead of the request being queued. The reason for this was predictability; the timespan of the radar capture is too long for buffering to be feasible.

On each script startup a telemetry log in the form of a .csv file was created from the program, named with the start date and time, so that a telemetry log never overwrote a previous log. Each row in the log held sixteen telemetry fields and five radar fields of data, in addition to the program runtime. Among the five radar fields was the file name of the radar capture (if one occurred), so that each radar capture could be matched directly to the telemetry rows.

To view live data for verification and testing, a terminal dashboard (which was viewed over SSH) showed flight state, GPS, attitude, battery voltage, the pulse width of the RC channel tied to the activation of the SDR, and the radar status. The display was refreshed at one Hz; a parameter changeable in the config file.

[Add screenshot of display]

*SDR Interface*

The SBC and radar did not communicate directly. Instead, the capture executable file was run by the `Radar` class in the program as a subprocess. This was to ensure appropriate separation between the two systems in order to minimise complexity. Under this method, the SBC was only responsible for scheduling and bookkeeping of the radar captures.

The capture settings for the radar existed in a JavaScript Object Notation (JSON) file which was read by the program at startup, so that parameters could change between flights without editing code. Among these settings were the executable and firmware paths, output folder for the radar captures, duration of capture, and sweep time.

[Capture sequence as a state machine. Will be changed in code so write here after that's done]

Each capture file was named according to its capture number and runtime on capture. Together with the per-row radar columns in the CSV file, this gave two independent ways to align the radar capture data with the telemetry data sets.

**Post-Processing**

A post-processing script that takes the SBC's synced .csv file and draws the flight path on an interactive map was created. The script is shown in Appendix A.5. When a new radar capture starts, a marker is placed on the interactive flight map at the location. The script erases the first row of data, as the longitude and latitude start at invalid value 0. This is because the flight controller initially outputs default values before beginning data recording. Coordinates are already in decimal degrees from the SBC program, so they are plotted directly.. The script is run on a desktop personal computer (PC), not the SBC. Data visualisation is done after capturing, so using a PC is appropriate. SDR data is not processed in this stage. An example output is shown in Figure 4.3.

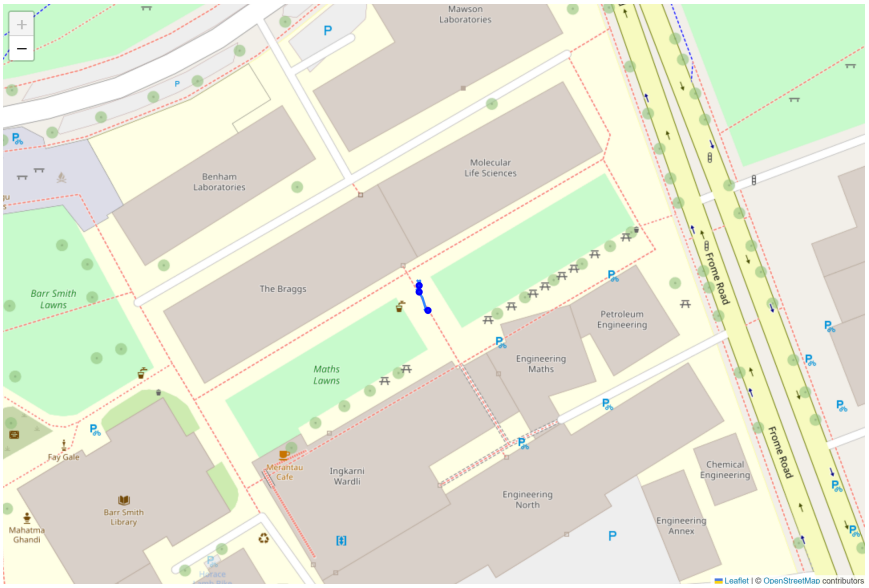



Figure 4.3: Flight path and radar capture locations from a test flight plotted in the data visualisation script.

The blue line is a point-to-point line between GPS data points, and the markers indicate points where radar data was captured. The markers also contain information about each radar capture (capture number, runtime and file name), and there is one marker per radar capture.

### 4.1.3 Results and Discussion

### 4.1.4 Summary

# Statement of AI Use

### Spelling and Grammar

The project team used ChatGPT (https://chat.openai.com/) and Claude (https://claude.ai/) for this Progress Report to check spelling and grammar in selected text drafted by the team. The use of AI tool was limited to identifying spelling and grammar errors. The tools were not used to draft new report sections, produce summaries or paraphrases, restructure the report, interpret results, draw conclusions. The project team reviewed all suggestions before making any changes and retained copies of the original text submitted to the AI tools.

The following prompt was used in May 2026:

> Please review the following paragraph and identify spelling and grammar errors only. Do not rewrite the text, change the meaning, improve the flow, shorten sentences, add content, or make style or substantive editing suggestions.

Initials of group members who used this support: HL, LT, KM, AS.

### Research Technical Support

The project team used Claude Code to analyse the legacy MATLAB driver, implement radar commands, and produce code documentation. All code and documentation generated by AI underwent human review and approval. AI use in this form was approved verbally by Dr Brian Ng. The following prompt was used in April 2026:

> Document all functions in the following file using Doxygen-style comments:
>
> `C:\Users\eliza\SDR_main\SDR_main\linux\src\UsbDevice.cpp`
>
> Documentation requirements:
>
> - Add a Doxygen comment block above every function definition.
> - Include:
>   - `@brief`
>   - `@param` for all parameters
>   - `@return` where applicable
>   - `@note` or `@warning` if relevant
> - Keep descriptions concise but technically clear.

Initials of group members who used this support: HL

# A Objective 4: Supporting Material

## A.1 SSH Setup

Listing A.1: The commands used to setup SSH connection between the Raspberry Pi and a remote computer.

```
sudo apt install openssh-server
sudo systemctl enable ssh
sudo ufw allow ssh
```

## A.2 GPIO Packages Install

Listing A.2: Bash command used to install python GPIO library.

```
sudo apt install python3-lgpio gpiod
```

## A.3 Virtual Environment and `rpi-lgpio` Install

Listing A.3: Commands used to create the python virtual environment and install `rpi-lgpio`.

```
wget http://abyz.me.uk/lg/lg.zip
unzip lg.zip
cd lg
make
sudo make install
python3 -m venv gpio-env
source gpio-env/bin/activate
pip install rpi-lgpio
```

## A.4 GPIO Read Script

Listing A.4: Python script used to run on the Raspberry Pi to monitor one of the GPIO pins.

```
#!/usr/bin/env python3
import RPi.GPIO as GPIO # Uses rpi-lgpio backend automatically
from time import sleep
import signal

# Setup
GPIO_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(GPIO_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

print("Sensing 3.3V High...")

try:
    while True:
        if GPIO.input(GPIO_PIN):
            print("Pin High")
            sleep(1)
except KeyboardInterrupt:
    print("Stopping...")
```

```
finally:
    GPIO.cleanup()
pause()
```

## A.5 Telemetry Data Visualisation Script

Listing A.5: Python script used to plot and visualise syncrhonised radar and telemetry data

```
import folium
import pandas as pd
import os

# parameters
filename = "flight_log_5.csv"


# Code
df = pd.read_csv(filename)

# first lat/lon sample is invalid, drop it and reindex from 0
df = df.iloc[1:].reset_index(drop=True)

output_file = os.path.splitext(filename)[0] + ".html" # -> "flight_log_4.html"
os.remove("synced_data.html") if os.path.exists("synced_data.html") else None

m = folium.Map(location=[df.at[0, "latitude"], df.at[0, "longitude"]], zoom_start=100)

# Create list of [latitude, longitude] points
points = list(zip(df["latitude"], df["longitude"]))

# Draw flight path
folium.PolyLine(points, weight=3).add_to(m)

capture_count = 0
for i, row in df.iterrows():
    tooltip_html = (
        f"<b>#{i}</b><br>"
        f"Latitude: {row['latitude']:.6f}<br>"
        f"Longitude: {row['longitude']:.6f}<br>"
        f"Capture Count: {row['radar_capture_count']}<br>"
    )

    if row["radar_capture_count"] == capture_count + 1:
        folium.CircleMarker(
            location=[row["latitude"], row["longitude"]],
            radius=3,
            color="blue",
            fill=True,
            fill_opacity=0.8,
            tooltip=tooltip_html,
        ).add_to(m)
        capture_count = 1 + capture_count
# Save map
```

```
46 m.save(output_file)
```